SESSION 21 / ORIGINAL CAPSTONE

Inventory, engineered for operations.

StockPilot

From a warehouse transaction
to a reproducible release.

MULTI-WAREHOUSE PROCUREMENT DEVSECOPS



Vimal Kumar Yadav · 24BCS10273

Original explanatory diagrams · synthetic English narration · deployment evidence stated explicitly

THE PRODUCT PROBLEM

One stock count. Many competing decisions.

01 / AVAILABILITY

Sell without overselling

Available stock must exclude reservations. A customer promise cannot be consumed by a warehouse transfer.

02 / CONSISTENCY

Move without losing stock

A transfer changes both warehouses in one transaction. Retries must not duplicate the movement.

03 / PROCUREMENT

Replenish with context

Low-stock signals connect the catalogue, supplier, purchase order and receiving workflow.

The complexity is in the rules, not just the number of services.

Example business workflows; capacity and production scale have not been benchmarked.

TECHNOLOGY CHOICES

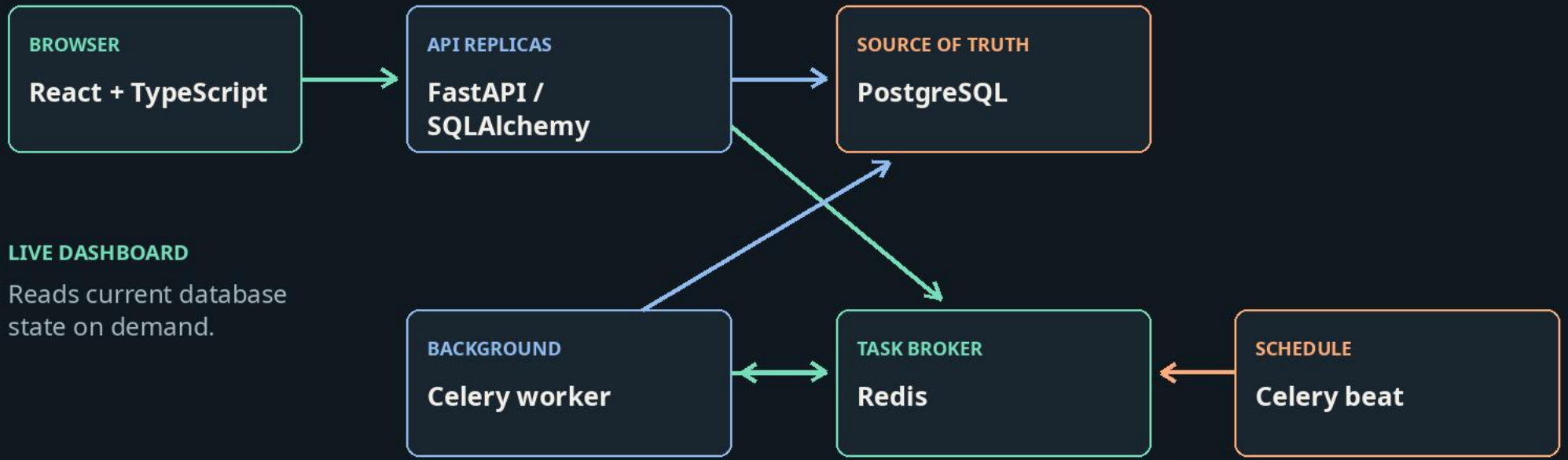
A different stack for each responsibility.

EXPERIENCE	**React · TypeScript · Vite** Typed forms and a responsive operational workspace
TRANSACTIONS	**FastAPI · SQLAlchemy · Alembic** Validated requests, business rules and schema migrations
DATA + ASYNC	**PostgreSQL · Redis · Celery** Durable stock ledger and separately scheduled reports
PLATFORM	**Docker · Kubernetes · Helm** Repeatable packaging, health checks and scaling controls
DELIVERY	**GitHub Actions · Argo CD · Terraform** Release gates, reconciliation and infrastructure definitions

Stack selection follows the instructor's Python/full-stack path; this is separate from the Java Typeahead project.

APPLICATION ARCHITECTURE

Two paths: serve requests and refresh reports.



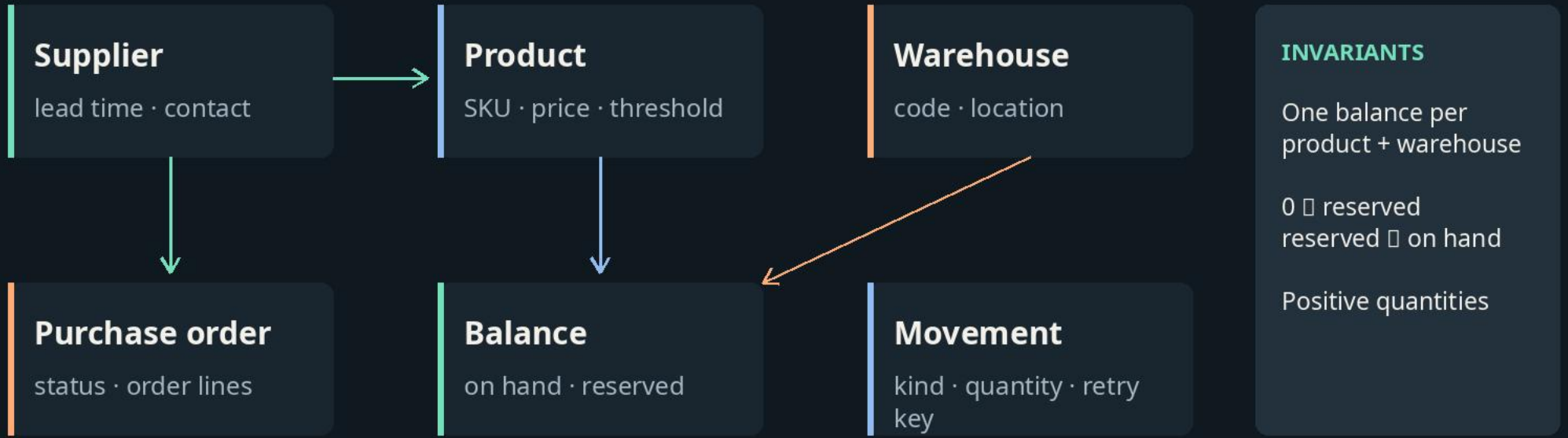
LIVE DASHBOARD

Reads current database state on demand.

Redis failure affects background reports; database-backed inventory endpoints remain a separate path.

DATA MODEL

Model the relationships before the screens.



INVARIANTS

One balance per product + warehouse

0 ≤ reserved ≤ on hand

Positive quantities

Additional entity: Reservation. Foreign keys protect referenced inventory history from accidental deletion.

A practical operations workspace.

Overview + inventory

Warehouse distribution, stock value, available units, low-stock signals and recent activity.

Products + suppliers

Searchable catalogue, categories, supplier associations, reorder thresholds and editable records.

Movements + reservations

Receive, issue or transfer stock; reserve units; release a promise or fulfil it into a stock issue.

Procurement

Create purchase orders and advance permitted transitions from draft to ordered to received.

Feature map based on application source; this diagram is not a captured browser screenshot.

A transfer is one transaction, not two edits.

Illustrative example: transfer 20 keyboards from Delhi to Bengaluru

DELHI / SOURCE

Before: 100 on hand
Reserved: 10
After: 80 on hand

TRANSFER 20 UNITS



140 total > 140 total

BENGALURU / TARGET

Before: 40 on hand
Reserved: 0
After: 60 on hand

Validate > lock product > check available > update both balances > append movement > commit

The numbers above illustrate the rule; real execution results are recorded separately in the evidence files.

A reservation changes availability first.

AVAILABLE = ON HAND − RESERVED



Closed reservations cannot switch to a conflicting final state.

Repeated completion of the same action is safe; insufficient available stock returns a conflict.

Receiving closes the procurement loop.

DRAFT

Editable supplier, warehouse and lines

→

ORDERED

Confirmed intent to purchase

→

RECEIVED

Atomic receipts for every order line

GUARDS

Supplier consistency · positive line quantities · valid transitions
Idempotent receiving · no edits after the draft stage

Draft and ordered purchases may be cancelled. Full-order receiving is supported; partial receipt is future work.

ASYNCHRONOUS PROCESSING

Background work must not own the stock ledger.

01 / REQUEST

TRIGGER

POST /api/reports
A scheduled beat every 60 seconds

02 / PROCESS

COMPUTE

Celery worker reads the current database snapshot

03 / SERVE

CACHE

Redis report + generated_at
180-second cache expiry

Graceful degradation

Report cache unavailable > explicit report error. Live dashboard > database-backed request.

Redis/Celery execution requires a runnable service environment; configuration alone is not a worker execution result.

Scale the API while protecting correctness.

Database coordination

PostgreSQL row locks serialize competing product changes. Stable lock ordering reduces deadlock risk.

Retry-aware writes

Request fingerprints distinguish an identical retry from a conflicting reuse of an idempotency key.

Independent replicas

API and frontend replicas can scale separately from workers. Shared durable state stays in PostgreSQL.

Honest capacity boundary

SQLite tests exercise rules, not PostgreSQL locking semantics. Load and concurrent database testing remain required.

Multiple replicas are an infrastructure configuration; production scalability requires measured capacity and failure tests.

CONTINUOUS INTEGRATION AND DELIVERY

Turn a source change into a gated release.

01
SOURCE
Lint + unit tests

02
SECURITY
SAST + dependencies

03
IMAGES
Build + image scan

04
DEPLOY
kind + Helm smoke

05
PROMOTE
GitOps image tag

TRACEABILITY

Commit SHA > container image > Helm value > deployed revision

A workflow file is not a successful GitHub Actions run.

CI definitions are reviewable locally. Hosted execution and GHCR publication require repository/network access.

RUNTIME PLATFORM

Kubernetes encodes operational expectations.

SERVE

Frontend + API Deployments
ClusterIP Services + Ingress
Startup, readiness and liveness probes
HPA + disruption budgets

PERSIST

PostgreSQL StatefulSet
Persistent volume claim
Redis for broker/cache
Migration job before rollout

CONSTRAIN

Non-root containers
Resource requests and limits
Network isolation policies
Configuration + external secrets

Readiness asks: can this replica safely receive traffic now?

A rendered chart describes intended resources. Running pods, persistence and failover need cluster evidence.

DEVSECOPS

Security has both release and runtime boundaries.

BEFORE RELEASE

Source analysis
Dependency vulnerability checks
Git history secret scanning
Container and infrastructure scans

DURING EXECUTION

Non-root workloads
Restricted capabilities
Explicit configuration and credentials
Network and resource boundaries

CURRENT PRODUCT BOUNDARY: no sign-in or RBAC; keep demo write APIs private.

A scan result is specific to a commit and vulnerability database. No passing scan is inferred from configuration.

Observe requests, then explain failures.

PROMETHEUS > GRAFANA

HTTP request counts
Request-duration histograms
Status labels and route labels
Dashboard and alert definitions

GIT > ARGO CD

Review desired state in Git
Reconcile the Helm release
Track the deployed image revision
Recover from configuration drift

JSON request logs connect method, route, status and elapsed time.

Live Grafana graphs and Argo CD Healthy/Synced status require captured runtime evidence.

TERRAFORM / DEPLOYMENT BOUNDARY

Cloud infrastructure is explicit and reviewable.

AWS VPC

PUBLIC SUBNETS

Default demo worker placement
Public IPs; NAT is optional

PRIVATE SUBNETS

Workers here with NAT enabled
Private subnets remain defined

Plan > review > apply > verify > capture > destroy

NO AWS DEPLOYMENT CLAIMED

AWS access is unavailable for this session. Definitions are separate from a real Terraform plan or apply.

VERIFICATION REGISTER

Evidence follows execution, not ambition.

Application code	**Implemented**	Catalog, stock rules, orders, reports
Backend verification	**40 passed · 1 skipped**	97% coverage; PostgreSQL concurrency pending
Frontend verification	**13 tests + build passed**	Component tests; browser capture pending
Containers + cluster	**Not executed here**	Local Docker socket unavailable
Browser recording	**Not available here**	Chromium sandbox process denied
Hosted CI + AWS	**Not claimed**	External access required

Read docs/evidence.md and the referenced logs for exact commands, scope and timestamps.

PRESENTER RUNBOOK / NEXT VERIFICATION

A complete demo has a clear next action.

PRODUCT WALKTHROUGH

1 Open dashboard and catalogue
2 Receive and transfer stock
3 Reserve, release and fulfil
4 Order and receive a purchase
5 Refresh a background report

PLATFORM WALKTHROUGH

1 Run backend and frontend checks
2 Build containers and install Helm
3 Verify persistence and recovery
4 Capture monitoring and GitOps
5 Plan/apply cloud only with access

Built to explain every stock change—and every release.

Use the runbook to demonstrate product workflows, delivery checks and operational recovery.